Informe de pruebas Servicios SOAP y REST

SoapUI (BCU Cotizaciones) · Postman (JSONPlaceholder Comments)

Tester: Diego de la Barrera

Índice

1. Objetivos
2. Pruebas ejecutadas
    2.1 Parte 1 - Servicio SOAP (BCU Cotizaciones)
        2.1.1 Descripción del servicio
        2.1.2 Observaciones durante las pruebas
        2.1.3 Casos de prueba válidos
        2.1.4 Assertions utilizadas y justificación — casos válidos
        2.1.5 Casos de prueba inválidos
        2.1.6 Assertions utilizadas y justificación - casos inválidos
    2.2 Parte 2 — Servicio REST (JSONPlaceholder Comments)
        2.2.1 Descripción del servicio
        2.2.2 Casos de prueba válidos
        2.2.3 Assertions (tests) utilizadas y justificación – casos válidos
        2.2.4 Casos de prueba inválidos
        2.2.5 Assertions (tests) utilizadas y justificación – casos inválidos
        2.2.6 Errores encontrados
3. Conclusiones
4. Uso de inteligencia artificial

1. Objetivos

El objetivo de este trabajo fue probar un servicio SOAP utilizando SoapUI y un servicio REST utilizando Postman. Para ambos servicios realicé casos de prueba válidos e inválidos, con el fin de verificar su funcionamiento y observar cómo manejan los errores.

En cada prueba registré el resultado esperado, el resultado obtenido y las validaciones (assertions) utilizadas para comprobar el comportamiento del servicio.

2. Pruebas ejecutadas

2.1 Parte 1 - Servicio SOAP (BCU Cotizaciones)

2.1.1 Descripción del servicio

Utilicé el servicio SOAP del Banco Central del Uruguay (BCU), que permite consultar cotizaciones de monedas.

WSDL: https://cotizaciones.bcu.gub.uy/wscotizaciones/servlet/awsbcucotizaciones?wsdl

La operación disponible es Execute, que recibe los siguientes parámetros:

- Moneda
- FechaDesde
- FechaHasta
- Grupo

Obtuve los códigos de moneda del archivo proporcionado por el BCU (por ejemplo, USD = 2222 y EUR = 1111).

2.1.2 Observaciones durante las pruebas

Antes de crear los casos de prueba realicé algunas pruebas manuales para entender el funcionamiento del servicio.

Pude observar que SoapUI genera automáticamente un comentario XML dentro del arreglo de monedas. Si ese comentario no se eliminaba antes de ejecutar el request, el servidor responde con un SOAP Fault, por lo que fue necesario borrarlo antes de ejecutar cada request.

También pude comprobar que el campo Grupo puede enviarse vacío sin generar errores.

2.1.3 Casos de prueba válidos

ID	DESCRIPCIÓN	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
TC01	Cotización dólar (USD 2222), fecha única 2026-05-06, Grupo en blanco	status=1, codigoerror=0, y devuelve la cotización del dólar para esa fecha	status=1, codigoerror=0, TCC=TCV?=39.868000, Nombre=DOLAR USA, CodigoISO=USD	**PASS**
TC02	Cotización Euro (código 1111), fecha única 2026-04-06.	status=1 y devuelve la cotización del Euro para esa fecha.	status=1. TCC=TCV=46.882493, Nombre=EURO, CodigoISO=EURO	**PASS**
TC03	Multi-moneda: USD (2222) + Euro (1111) en un mismo request, fecha 2026-05-06.	status=1 y devuelve 2 registros, uno por cada moneda solicitada	status=1, 2 registros (EURO y USD). El orden de respuesta no coincide con el orden de envío.	**PASS**
TC04	Moneda=0 (comodín, trae todas las cotizaciones), fecha 2026-05-06.	status=1 y devuelve el listado completo de monedas cotizados ese día.	status=1, 40 registros (incluye monedas, unidades indexadas UI/UP/UR y tasa de mercado).	**PASS**
TC05	Rango de fechas 2026-05-04 a 2026-05-08 (5 días), USD.	status=1 y devuelve un registro por cada día del rango solicitado.	status=1, 5 registros (uno por día), con TCC distinto en cada fecha.	**PASS**

2.1.4 Assertions utilizadas y justificación — casos válidos

- **TC01 – Cotización USD (fecha única)**
- **Not SOAP Fault:** Utilicé para comprobar que el request fue procesado correctamente y que el servicio no devolvió un error SOAP.
- **Contains <status>1</status>:** Utilicé para verificar que la consulta fue exitosa.
- **XPath Match sobre //*:TCC = 39.868000:** Utilicé para validar el valor exacto de la cotización del dólar.

- TC02 – Cotización Euro (fecha única)

- **Not SOAP Fault**
- **Contains <status>1</status>**
- **XPath Match sobre //*:TCC = 46.882493**

Utilicé las mismas assertions que en el caso anterior para comprobar que el servicio devolviera correctamente la cotización del euro.

- **TC03 – Consulta de USD y Euro**
- **XPath Match: count(//*:datoscotizaciones.dato) = 2:** Utilicé para verificar que el servicio devolviera dos registros.
- **Contains <CodigoISO>USD</CodigoISO>**
- **Contains <CodigoISO>EURO</CodigoISO>**

Estas assertions permitieron comprobar que estaban presentes las dos monedas solicitadas.

- **TC04 – Consulta de todas las monedas**
- **XPath Match: count(...) = 40**

Utilicé esta validación para comprobar la cantidad de registros devueltos por el servicio al consultar todas las monedas.

TC05 – Consulta por rango de 5 días

- **XPath Match: count(...) = 5**

Utilicé para verificar que el servicio devolviera un registro por cada día del rango consultado.

2.1.5 Casos de prueba inválidos

ID	DESCRIPCIÓN	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
TC06	Código de moneda inexistente (9999), fecha 2026-05-06.	El servicio debe rechazar el código y reportarlo mediante un código de error de negocio.	status=0, codigoerror=101, mensaje="Código de moneda no existe". Sin SOAP Fault.	**PASS**
TC07	Rango de fechas invertido: FechaDesde=2026-05-08 posterior a FechaHasta=2026-05-04	El servicio debe detectar el rango inválido y reportarlo con un código de error específico.	status=0, codigoerror=103, mensaje="Fecha Hasta es menor que Fecha Desde".	**PASS**
TC08	Formato de fecha incorrecto: "06/05/2026" DD/MM/AAAA en vez de AAAA-MM-DD.	El servicio debe validar el formato y reportar un error.	status=0, codigoerror=102, mensaje="Campo de fecha inválida".	**PASS**

TC09	[Caso de borde] Rango de fechas de 40 días corridos (2026-03-27 a 2026-05-06), y la totalidad de monedas	El servicio debería rechazar rangos mayores a 31 días con más de 10 monedas	status=0, codigoerror=104, mensaje="Rango de fechas excede lo permitido". Confirma que el límite existe pero se dispara según el volumen de datos	**PASS**
TC10	Se omite la etiqueta e fecha <cot:FechaDesde>	Confirmar si el servicio trata el campo como obligatorio pese a ser opcional	status=0, codigoerror=102 (mismo código que fecha con formato inválido). Confirma que es obligatorio, aunque en la página lo permita omitir.	**PASS (hallazgo)**
TC11	Código de moneda no numérico: envío el texto USD en vez de un entero.	El servicio debería validar el tipo de dato y responder con un error.	HTTP 500 Server Error, sin SOAP Fault ni codigoerror.	**BUG**

2.1.6 Assertions utilizadas y justificación - casos inválidos

- **TC06, TC07, TC08, TC09 y TC10**

En estos casos probé diferentes errores de entrada, como una moneda inexistente, un rango de fechas invertido, un formato de fecha incorrecto, un rango mayor al permitido y una fecha omitida.

Las assertions que utilicé fueron:

- **Not SOAP Fault:** para verificar que el servicio respondiera correctamente y que el error fuera manejado por el sistema, sin generar un error SOAP.
- **Contains <status>0</status>:** para comprobar que el servicio detectó que los datos enviados eran inválidos.
- **Contains <codigoerror>:** para validar el código de error correspondiente a cada caso (101, 102, 103 o 104).
- **Contains del mensaje de error:** para verificar que el mensaje devuelto coincidiera con el error esperado.

Estas validaciones me permitieron comprobar que el servicio manejó correctamente los errores de negocio y devolvió la respuesta esperada.

- **TC11 – Moneda no numérica**

En este caso envié **USD** como texto en lugar de un código numérico.

Las assertions utilizadas fueron:

- **Valid HTTP Status Codes (esperado: 200):** para comprobar que el servicio respondiera correctamente. La prueba falló porque el servidor devolvió un error 500.
- **Contains "Server Error":** para confirmar que la respuesta correspondía a un error interno del servidor.

Este caso me permitió detectar un problema en el servicio, ya que no manejó correctamente un tipo de dato inválido y respondió con un **HTTP 500 Internal Server Error**.

2.2 Parte 2 — Servicio REST (JSONPlaceholder Comments)

2.2.1 Descripción del servicio

Para la parte REST utilicé lla API pública JSONPlaceholder, específicamente el recurso /comments. Esta API se usa con fines de prueba y no guarda los cambios realizados sobre los datos.

Base URL: https://jsonplaceholder.typicode.com

Recurso probado: /comments

Realicé pruebas sobre las siguientes operaciones:

- GET: obtener comentarios (todos, por id y filtrados por parámetros).
- POST: crear un comentario.
- PUT: actualizar un comentario.
- DELETE: eliminar un comentario.

La estructura incluye los siguientes campos:

- postId (entero)
- id (entero)
- name (texto)
- email (texto)
- body (texto)

Una característica importante de esta API es que las operaciones POST, PUT y DELETE no modifican realmente la información del servidor. Los cambios solo se reflejan en la respuesta que devuelve la API durante esa ejecución.

2.2.2 Casos de prueba válidos

ID	DESCRIPCIÓN	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
GET-Válido1	GET /comments/45, búsqueda por ID específico.	200 OK, devuelve el objeto comentario con id=45.	200 OK, devuelve el objeto comentario con id=45.	**PASS**
GET-Válido2	GET /comments?postId=9.	200 OK, devuelve únicamente los comentarios cuyo postId sea 9.	200 OK, devuelve únicamente los comentarios cuyo postId sea 9.	**PASS**
POST-Válido 1	POST /comments, crear comentario nuevo (postId=9 + name/email/body).	201 Created, devuelve el comentario creado con un id autogenerado.	201 Created. id=501 asignado automáticamente, demás de campos reflejados igual a lo enviado.	**PASS**
POST-Válido 2	POST /comments, segundo caso, con postId=1 (variación de datos).	201 Created, devuelve el comentario creado con un id autogenerado.	201 Created. id=501 (igual al caos anterior), confirma que no hay persistencia real en el sistema.	**PASS**
PUT-Válido1	PUT /comments/45, actualizar comentario existente (todos los campos menos id).	200 OK, devuelve el recurso actualizado con el id tomado de la URL.	200 OK. id=45 (tomado de la URL). Campos actualizados según lo enviado.	**PASS**
PUT-Válido2	PUT /comments/1, segundo caso de actualización, con otro id.	200 OK, devuelve el recurso actualizado con el id=1 tomado de la URL.	200 OK. id=1, campos actualizados correctamente.	**PASS**
DEL-Válido1	DELETE /comments/45, borrar comentario existente.	200 OK confirmando el borrado.	200 OK, body = objeto vacío {}. No confirma directamente qué se borró ni incluye mensaje.	**PASS**
DEL-Válido2	DELETE /comments/1, segundo caso de borrado, con otro id.	200 OK, mismo comportamiento que el anterior	200 OK, body = objeto vacío {}. Mismo comportamiento que el casos anterior.	**PASS**

2.2.3 Assertions (tests) utilizadas y justificación – casos válidos

- **GET-Válido1 y GET-Válido2 – Búsqueda por ID**

- **Status code 200:** Utilicé para comprobar que la solicitud fue exitosa.
- **Comparación de campos (id, postId, email, etc.):** Verifiqué que los datos devueltos coincidieran con los esperados.
- **Validación de las claves del objeto:** Comprobé que la respuesta incluyera todos los campos esperados.

En el caso **GET-Válido2**, además verifiqué que todos los comentarios obtenidos tuvieran **postId = 9**.

- **POST-Válido1 y POST-Válido2 – Creación de comentarios**

- **Status code 201:** Verifiqué que el comentario fue creado correctamente.
- **Comparación de los datos enviados:** Comprobé que la respuesta devolviera los mismos valores enviados en el request.
- **Validación del id (501):** verifiqué que el id devuelto por la API fue el mismo en ambas pruebas.

- **PUT-Válido1 y PUT-Válido2 – Actualización de comentarios**

- **Status code 200:** Comprobé que la actualización fue exitosa.
- **Validación del id:** Verifiué que el id de la respuesta coincidiera con el id enviado en la URL.
- **Comparación de los campos actualizados:** Comprobé que los datos modificados fueran los esperados.

- **DEL-Válido1 y DEL-Válido2 – Eliminación de comentarios**

- **Status code 200:** Verifiqué que la operación se ejecutó correctamente.
- **Validación del body vacío:** Comprobé que la respuesta devolviera un objeto vacío.

Estas assertions me permitieron verificar que cada operación respondiera correctamente y que los datos devueltos fueran los esperados.

2.2.4 Casos de prueba inválidos

ID	DESCRIPCIÓN	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
GET-Inválido 1	GET /comments/505, id fuera del rango existente (1-500).	404 Not Found, con un mensaje descriptivo del error.	404 Not Found. Body = objeto vacío {}, sin mensaje descriptivo del error.	**PASS (faltó mensaje de error)**
POST-Inválido1	POST /comments con postId=9999 (no existe en /posts, solo debería ir hasta 100)	El servicio debería validar y rechazar el postId inexistente.	201 Created. Acepta el postId=9999 sin ninguna validación, reflejando todos los campos igual que un caso válido.	**OBSERVACIÓN (sin validación)**
PUT-Inválido 1	PUT /comments/9999, actualizar un id que no existe en la colección.	El servicio debería responder 404 (no encontrado) o rechazar la operación.	HTTP 500 Internal Server Error.	**BUG**
DEL-Inválido 1	DELETE /comments/9999, borrar un id que no existe en la colección.	El servicio debería responder 200 vacío o 404, sin exponer errores internos.	200 OK, body = objeto vacío {}. Mismo comportamiento que borrar un id existente, no distingue si había algo para borrar.	**OBSERVACIÓN**

2.2.5 Assertions (tests) utilizadas y justificación – casos inválidos

- **GET-Inválido1 – ID inexistente (505)**

- **Status code 404:** Utilicé para comprobar que el servicio respondiera con un error cuando el comentario no existe.
- **Body vacío:** Verifiqué que la respuesta devolviera un objeto vacío ({}).

Estas validaciones permitieron comprobar que el servicio respondió correctamente ante un id inexistente.

- **POST-Inválido1 – postId inexistente (9999)**

- **Status code 201:** Utilicé para verificar la respuesta del servicio.
- **Validación del postId y de los demás campos:** Comprobé que la API devolviera los mismos datos enviados.

En este caso se pudo observar que el servicio acepta un **postId** que no existe y crea la respuesta igualmente.

- **PUT-Inválido1 – Actualizar un ID inexistente (9999)**

- **Status code 500:** Utilicé para comprobar el código de error obtenido.
- **Contains “TypeError”:** Verifiqué que el mensaje de la respuesta correspondiera a un error interno del servidor.

Estas assertions me permitieron confirmar que el servicio no manejó correctamente este caso.

DEL-Inválido1 – Borrar un ID inexistente (9999)

- **Status code 200:** Utilicé para comprobar la respuesta del servicio.
- **Body vacío:** Verifiqué que la respuesta fuera un objeto vacío ({}).

Con esta prueba pude observar que la API responde igual cuando el id existe y cuando no existe.

2.2.6 Errores encontrados

El problema más importante fue el caso **PUT-Inválido1**, donde al intentar actualizar un comentario con un id inexistente el servicio respondió con un **HTTP 500 Internal Server Error**.

Además, la respuesta mostraba un mensaje de error interno **TypeError**. Esto indica que el servicio no valida correctamente si el recurso existe antes de intentar actualizarlo.

También se observó que cada operación maneja de forma diferente un id inexistente:

- **GET** devuelve 404 Not Found.
- **POST** acepta un postId inexistente y responde 201 Created.
- **PUT** devuelve 500 Internal Server Error.
- **DELETE** devuelve 200 OK con un objeto vacío.

Esto muestra que la API no tiene un comportamiento consistente para manejar recursos inexistentes.

3. Conclusiones

Este trabajo me permitió aplicar los conceptos vistos durante el curso utilizando dos herramientas diferentes: SoapUI para probar un servicio SOAP y Postman para probar un servicio REST.

En el servicio SOAP del BCU, la mayoría de las pruebas funcionaron correctamente y el servicio respondió de forma adecuada en los casos válidos y en varios casos inválidos. Se comprobó que maneja distintos errores mediante códigos específicos (101, 102, 103 y 104). Por otro lado, también se encontró un problema cuando se envió un código de moneda con un tipo de dato incorrecto, ya que el servicio respondió con un HTTP 500 Internal Server Error.

En el servicio REST de JSONPlaceholder se observó que, al ser una API de prueba, algunas validaciones no se realizan. Por ejemplo, permite crear comentarios con un postId inexistente y, en algunos casos, responde de forma diferente según la operación realizada. El caso más importante fue el PUT sobre un recurso inexistente, que también generó un error 500.

Una de las cosas más importantes que se pudo comprobar es que las pruebas inválidas y los casos de borde son muy útiles para detectar errores. Si solo se probaran los casos correctos, varios de los problemas encontrados no habrían aparecido.

El uso de assertions me permitió automatizar la verificación de las respuestas y comparar el resultado esperado con el resultado obtenido. En lugar de verificar únicamente que la solicitud no fallara, se comprobaron valores específicos como el código de estado, los códigos de error, los datos devueltos y la cantidad de resultados.

En conclusión, el trabajo me permitió practicar la creación de casos de prueba, el uso de SoapUI y Postman, y el análisis de respuestas de servicios. Además, me ayudó a entender la importancia de probar tanto los escenarios correctos como los incorrectos para poder detectar errores y documentarlos adecuadamente.

4. Uso de inteligencia artificial

ITEM	DESCRIPCIÓN
HERRAMIENTA	ChatGPT
PROPÓSITO	Se utilizó como apoyo para comprender mejor el funcionamiento de algunas Assertions que aún no había utilizado, resolver y evacuar algunas dudas referente a respuestas y creación de scripts en las validaciones de Postman, y mejorar la redacción y organización del informe.
DECISIONES AUTÓNOMAS	La selección de los casos de prueba fue realizada por mí. La ejecución de las pruebas, la configuración de las requests y de las assertions también fue realizada por mi con el apoyo de ChatGPT mencionado en el punto anteiror. El análisis de los resultados obtenidos, se realizaron a partir del material proporcionado por CES, las clases del curso y la experimentación realizada durante la práctica.